

FEC and LDPC Internal Decoder Update

Date: 2026-06-05

Summary

The external decoder dependencies for convolutional FEC, CCSDS Reed-Solomon, and binary LDPC decoding were replaced with internal C99-compatible implementations.

Public decoder APIs were preserved:

- `sdr_decode_conv()`
- `sdr_decode_rs()`
- `sdr_decode_LDPC()`
- `sdr_decode_NB_LDPC()`

Build dependencies removed from the normal build path:

- `libfec.a`
- `libldpc.a`
- `lib/build/libfec.mk`
- `lib/build/libldpc.mk`
- `LDPC-codes` / `libfec` clone steps

The non-binary LDPC decoder in `src/sdr_nb_ldpc.c` was already internal and was left unchanged.

Implemented Algorithms

Viterbi Decoder

Implemented in `src/sdr_fec.c`.

- Code: convolutional code, $K=7$, $R=1/2$
- Polynomials: `G1=0x4F`, `G2=0x6D`
- Trellis: 64 states
- Input metric: `uint8_t` soft symbols in the range 0 to 255
- Hard decision remains supported by using 0 and 255 symbols
- Branch metric uses the received soft-symbol confidence directly
- Chainback starts from state 0, matching the existing zero-tail usage

This is a straightforward full-trellis Viterbi implementation. It favors simple code over SIMD or packed-path optimizations.

Reed-Solomon Decoder

Implemented in `src/sdr_fec.c`.

- Code: CCSDS RS(255,223)
- Field: GF(256), primitive polynomial `0x187`
- Generator parameters: `FCR=112`, `PRIM=11`, `NROOTS=32`
- CCSDS dual-basis conversion is generated internally
- Decoder flow:
 - syndrome computation
 - Berlekamp-Massey error-locator search
 - Chien search
 - Forney error-value correction
 - final syndrome validation

The return value remains the number of corrected error bits, or `-1` for uncorrectable input.

Binary LDPC Decoder

Implemented in `src/sdr_ldpc.c`.

- The old `mod2sparse` matrix API was removed.
- H matrices are now generated into an internal Tanner graph:
 - row adjacency list
 - column adjacency list
 - edge row/column arrays
- Decoder: normalized min-sum belief propagation
- Max iterations: `250`
- Initial hard-decision LLR magnitude: `10.0`
- Min-sum normalization scale: `0.75`
- Input polarity matches the previous `LDPC-codes` behavior.

This decoder is used for:

- `CNV2_SF2`
- `CNV2_SF3`
- `IRNV1_SF2`
- `IRNV1_SF3`

BDS `BCNV1_SF2`, `BCNV1_SF3`, `BCNV2`, and `BCNV3` still use the existing internal non-binary LDPC decoder.

Verification

Commands run:

```
gcc -std=c99 -Wall -Wextra -Isrc -Ilib/RTKLIB/src -c src/sdr_fec.c -o /tmp/sdr_fec.o
gcc -std=c99 -Wall -Wextra -Isrc -Ilib/RTKLIB/src -c src/sdr_ldpc.c -o /tmp/sdr_ldpc.o
make -B -C test/utest test_sdr_fec USE_AVX2=0
make -B -C test/utest test_sdr_ldpc USE_AVX2=0
make -B -C test/utest test_sdr_nav USE_AVX2=0
make -B -C lib/build -f libshr.mk libshr.a USE_SOAPY=0 USE_AVX2=0
```

All related unit tests passed.

Speed Comparison

Environment:

- WSL
- `gcc -O3`
- Old implementations linked directly from existing local `lib/linux/libfec.a` and `lib/linux/libldpc.a`

FEC Speed

Decoder	Internal	Old library	Result
Viterbi, 258-bit frame	53.25 us/call	15.23 us/call	internal is about 3.5x slower
RS(255,223), 8 symbol errors	53.83 us/call	11.29 us/call	internal is about 4.8x slower

The slower FEC result is expected. These implementations are intentionally simple and do not use SIMD or packed path metrics.

Binary LDPC Speed

Decoder	Internal	Old LDPC-codes	Result
<code>CNV2_SF3</code> , 548 bits, 5 bit errors	10.7 us/call	22.0 us/call	internal is about 2.0x faster
<code>IRNV1_SF3</code> , 548 bits, 4 bit errors	10.0 us/call	21.0 us/call	internal is about 2.1x faster

The binary LDPC internal decoder is faster because it uses a fixed Tanner graph and normalized min-sum updates instead of the old generic sparse-matrix API.

Bit-Error Injection Results

All tests used fixed random seeds and identical random error patterns for the internal and old decoders.

Viterbi

Input: 258 information bits plus 6 zero tail bits. Errors are random flips in the encoded 528-symbol stream. Success means the 258 decoded bits match the original message.

Trials: 1000

Encoded symbol flips	Internal	Old libfec
0	100.0%	100.0%
2	100.0%	100.0%
4	100.0%	100.0%
6	100.0%	100.0%
8	100.0%	99.9%
10	100.0%	100.0%
12	100.0%	100.0%
16	99.4%	99.3%
20	98.6%	98.1%
24	95.0%	93.8%
32	80.5%	77.8%
40	51.7%	47.3%
48	14.4%	11.7%
64	0.0%	0.0%

The internal Viterbi decoder is effectively equivalent to the old libfec decoder for this hard-error model, with slightly better results in this sample.

Reed-Solomon

Input: CCSDS RS(255,223) codeword. Errors are random bit flips across the full 255-byte codeword. Success means the corrected codeword exactly matches the original codeword.

Trials: 1000

Codeword bit flips	Internal	Old libfec
0	100.0%	100.0%
2	100.0%	100.0%
4	100.0%	100.0%
8	100.0%	100.0%
12	100.0%	100.0%
16	100.0%	100.0%
17	39.2%	39.2%
18	9.5%	9.5%
19	1.9%	1.9%
20	0.2%	0.2%
24	0.0%	0.0%
32	0.0%	0.0%

Codeword bit flips	Internal	Old libfec
40	0.0%	0.0%
48	0.0%	0.0%
64	0.0%	0.0%
80	0.0%	0.0%
96	0.0%	0.0%
128	0.0%	0.0%

The internal RS decoder matches old libfec exactly in this test. The sharp drop above 16 bit flips is expected because random bit flips typically affect more than 16 distinct RS symbols, and RS(255,223) corrects up to 16 symbol errors.

Binary LDPC CNV2_SF3

Input: 548-bit **CNV2_SF3** test codeword. Errors are random bit flips. Success means the decoded 274 message bits match the original message.

Trials: 1000

Bit errors	Internal	Old LDPC-codes
0	100.0%	100.0%
2	100.0%	100.0%
4	100.0%	100.0%
6	100.0%	100.0%
8	100.0%	100.0%
10	100.0%	100.0%
12	100.0%	100.0%
14	100.0%	100.0%
16	100.0%	100.0%
20	100.0%	100.0%
24	99.9%	99.8%
32	99.9%	100.0%
40	97.1%	97.0%
48	67.3%	5.8%
64	0.0%	0.0%
80	0.0%	0.0%

Binary LDPC IRNV1_SF3

Input: 548-bit **IRNV1_SF3** test codeword. Errors are random bit flips. Success means the decoded 274 message bits match the original message.

Trials: 1000

Bit errors	Internal	Old LDPC-codes
0	100.0%	100.0%
2	100.0%	100.0%
4	100.0%	100.0%
6	100.0%	100.0%
8	100.0%	100.0%
10	100.0%	100.0%
12	99.8%	100.0%
14	99.9%	99.9%
16	99.9%	100.0%
20	99.4%	99.8%
24	98.8%	99.9%
32	95.2%	98.8%
40	77.3%	93.3%
48	34.4%	40.9%
64	0.0%	0.0%
80	0.0%	0.0%

Notes

- The FEC decoders are slower than libfec, but the absolute decode cost is small for navigation-message workloads.
- The binary LDPC internal decoder is faster than the old library in the tested cases.
- Binary LDPC high-error performance differs by code type. **CNV2_SF3** improved in the tested high-error range, while **IRNV1_SF3** is somewhat weaker than the old probability-propagation decoder near the waterfall region.
- Normal operating conditions are expected to be well below the high-error cases where the LDPC differences become significant.

SBAS Viterbi Compatibility Fix

After the initial internal Viterbi replacement, SBAS L1C/A navigation data decoding was less stable than the old **libfec**-based decoder. The receiver showed frequent SBAS frame-lock loss even though GPS L1C/A tracking remained normal.

The cause was a behavioral difference between the first internal Viterbi implementation and **libfec**:

- The first implementation used a generic 64-state max-metric trellis.
- **libfec** uses a specific butterfly update and chainback convention.
- With no errors, both decoders produced the same output.
- With noisy SBAS symbols, tie-breaking and path-selection differences could produce different decoded navigation bits.

The Viterbi decoder in `src/sdr_fec.c` was updated to match the old `libfec` portable decoder behavior:

- branch table generation matches `set_viterbi27_polynomial_port()`
- butterfly metric update matches `update_viterbi27_blk_port()`
- decision storage follows the same 64-state decision convention
- chainback starts after the 6 tail bits, matching `chainback_viterbi27_port()`

Compatibility Test

The fixed internal decoder was compared against the old `libfec` Viterbi decoder using the same encoded test frames and the same random symbol-flip patterns.

Environment:

- MSYS2 UCRT64
- `gcc -O3`
- old decoder linked from `lib/win32/libfec.a`

Result:

Encoded symbol flips	Patterns	Output differences
0	10000	0
8	10000	0
16	10000	0
24	10000	0
32	10000	0
40	10000	0
48	10000	0

Related UCRT64 verification:

```
make -B -C test/utest test_sdr_fec USE_AVX2=0
cd test/utest && ./test_sdr_fec.exe

make -B -C test/utest test_sdr_nav USE_AVX2=0
cd test/utest && ./test_sdr_nav.exe

make -B -C lib/build -f libsdr.mk libsdr.a USE_SOAPY=0 USE_AVX2=0
```

After this fix, SBAS L1C/A navigation data decoding stability was confirmed to improve in receiver operation.